

FakeNewsChecker
Object Design Document
Versione 1.1



**FakeNews
Checker**

Data: 11/01/2026

Progetto: FakeNewsChecker	Versione: 1.1
Documento: Object Design Document	Data: 11/01/2026

Coordinatore del progetto:

Nome	Matricola

Partecipanti:

Nome	Matricola
De Simone Federica	0512120472
Tirelli Giuseppe	0512120367
Falasca Swami	0512119323

Scritto da:	Swami Falasca, De Simone Federica, Tirelli Giuseppe
--------------------	---

Revision History

Data	Versione	Descrizione	Autore
20/12/2025	1.0	Prima versione dell'Object Design Document	Swami Falasca, De Simone Federica, Tirelli Giuseppe
11/01/2026	1.1	Seconda versione, modifica dei ruoli	Swami Falasca, De Simone Federica, Tirelli Giuseppe

Indice

1. INTRODUZIONE	4
1.1. OBJECT DESIGN TRADE-OFFS	4
1.1.1. Tempo di rilascio VS Funzionalità	4
1.1.2. Velocità VS Memoria	4
1.1.3. Costruire VS Comprare	4
1.2. OFF-THE SHELF COMPONENTS	4
1.3. INTERFACE DOCUMENTATION GUIDELINES	5
1.4. DEFINITIONS, ACRONYMS AND ABBREVIATIONS	6
1.5. REFERENCES	6
2. PACKAGES	6
2.1. SUBSYSTEM DECOMPOSITION	6
2.1.1. Interface	7
2.1.2. Application logic	8
2.1.3. Storage	8
3. CLASS INTERFACES	9
3.1. UtenteDAO	9
3.2. NotiziaDAO	10
3.3. SegnalazioneDAO	12
3.4. GestoreDAO	14

1. INTRODUZIONE

1.1. *OBJECT DESIGN TRADE-OFFS*

1.1.1. *Tempo di rilascio VS Funzionalità*

Per garantire un sistema completo e affidabile per i clienti, abbiamo scelto di privilegiare la completezza delle funzionalità rispetto al tempo di rilascio.

1.1.2. *Velocità VS Memoria*

Utilizzare più memoria per ottimizzare l'accesso veloce ai dati può aumentare le prestazioni, anche se potrebbe richiedere risorse aggiuntive.

1.1.3. *Costruire VS Comprare*

Nonostante l'uso di software già disponibile offra vantaggi come l'accesso a funzionalità pronte e un carico di lavoro ridotto per gli sviluppatori, si è preferito costruire la maggior parte del Sistema da zero, ricorrendo a componenti esterne solo in circostanze specifiche.

Trade-Off	
Tempo di rilascio	Funzionalità
Velocità	Memoria
Costruire	Comprare

1.2. *OFF-THE SHELF COMPONENTS*

Il Sistema utilizzerà i seguenti componenti off-the-shelf:

1. **JQuery**: framework ampiamente utilizzato per semplificare la scrittura di codice JavaScript.
2. **MySQL**: sistema open source di gestione di database relazionali SQL.
3. **MySQL Connector/J (JDBC driver)**: driver JDBC ufficiale per la comunicazione tra Java e MySQL.
4. **Tomcat**: Web Server e application container per applicazioni Java.

1.3. *INTERFACE DOCUMENTATION GUIDELINES*

Queste linee guida definiscono una serie di norme che il team di sviluppo deve rispettare durante la progettazione del sistema. Il progetto FakeNews Checker è sviluppato utilizzando l'ambiente IntelliJ IDE e presenta la seguente organizzazione:

- **Package:** i nomi dei pacchetti devono essere espressi esclusivamente in caratteri minuscoli.
- **Classi:** la denominazione delle classi deve essere significativa e aderire alla convenzione UpperCamelCase.
- **Interfacce:** anche le interfacce seguono la convenzione UpperCamelCase per la scelta del nome.
- **Metodi:** i nomi dei metodi devono conformarsi allo stile camelCase.
- **Variabili:** la scelta del nome per le variabili deve essere esplicativa. Sono da evitare denominazioni composte da singole lettere, fatta eccezione per variabili temporanee.

Indentazione:

- Le istruzioni contenute in un blocco (ad esempio, un ciclo for) devono essere indentate di un livello rispetto all'istruzione composta.
- La parentesi graffa di apertura deve essere posizionata alla fine della riga dell'istruzione composta.
- La parentesi graffa di chiusura deve allinearsi orizzontalmente con il livello di indentazione dell'istruzione composta.

HTML: per lo sviluppo HTML è stato adottato il riferimento:

https://www.w3schools.com/html/html5_syntax.asp

CSS: gli stili non in-line devono essere definiti esclusivamente in fogli di stile esterni.

JavaScript:

- Il codice JavaScript deve essere organizzato in file dedicati.
- Per l'impaginazione e la denominazione degli elementi, il codice JavaScript deve uniformarsi alle convenzioni stabilite per Java.

Database SQL: la denominazione di tabelle e campi deve rispettare queste regole:

- Utilizzare esclusivamente lettere minuscole. In caso di nomi composti da più parole, separare i termini mediante underscore (_).

1.4. DEFINITIONS, ACRONYMS AND ABBREVIATIONS

- **JQuery**: framework utilizzato per semplificare la scrittura di codice JavaScript
- **MySQL**: sistema di gestione di database relazionali SQL
- **MySQL Connector/J (JDBC driver)**: driver JDBC ufficiale che consente e semplifica la comunicazione tra applicazioni Java e il database MySQL.
- **SQL**: Structured Query Language
- **JDBC**: Java DataBase Connectivity
- **HTML**: HyperText Markup Language
- **CSS**: Cascading Style Sheets

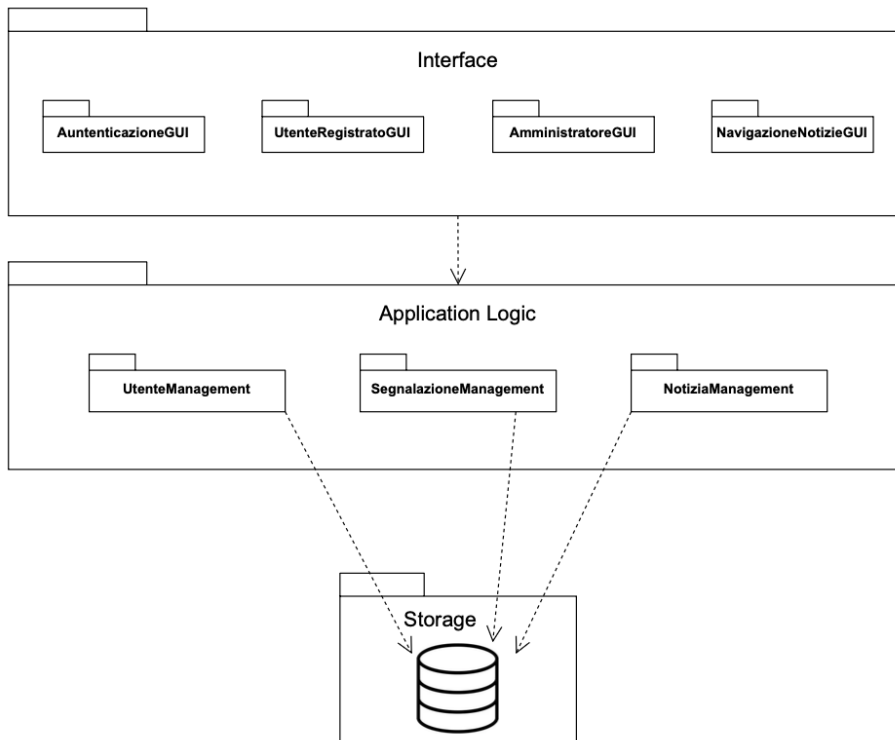
1.5. REFERENCES

Per stilare questo documento, è stato preso come riferimento il libro di testo “Object-Oriented Software Engineering Using UML, Patterns and Java: Third Edition, di Bernd Bruegge ed Allen H. Dutoit”.

2. PACKAGES

In questa parte viene presentata l'organizzazione in package del Sistema, come definito nel progetto di System Design. La ripartizione è conseguenza delle scelte architetturali effettuate e evidenzia l'aderenza alle strutture delle directory di Maven.

2.1. SUBSYSTEM DECOMPOSITION



2.1.1. Interface

La struttura del package si articola nei seguenti sub-package e classi:

- Package AutenticazioneGUI
 - LoginFailed
 - Login
 - LoginGestore
 - LoginGestoreFailed
 - Register
- Package UtenteRegistratoGUI
 - AreaUtente
 - ConfermaSegnalazione
 - StoricoSegnalazioni
- Package GestoreTecnicoGUI
 - Dashboard
- Package GestoreVerificheGUI
 - Dashboard
 - elencoGestori
 - gestioneArticoli
 - inserisciArticoli
 - registraGestore
 - successoRegistrazioneGestore
- Package NavigazioneNotiziaGUI
 - dettaglioNotizia
 - risultatiRicerca
 - visualizzaNotizie
- Package SegnalazioneNotiziaGUI
 - errore
 - formSegnalazione
 - storicoSegnalazioni
 - successoSegnalazione
- Package UtenteRegistratoGUI
 - UserProfile

Sono inoltre definite le interfacce HomePage, Header e Footer.

2.1.2. Application logic

La struttura del package si articola nei seguenti sub-package e classi:

- Package AutenticazioneManagement
 - LoginServlet
 - LogoutGestoreServlet
 - LogoutUtenteServlet
 - RegisterUtenteServlet
- Package GestoreTecnicoManagement
 - GestoreTecnicoDashboardServlet
- Package GestoreVerificheManagement
 - AggiornaVerificaServlet
 - ElencoGestoriServlet
 - GestioneArticoliServlet
 - GestoreVerificheDashboardServlet
 - InserisciArticoloServlet
 - RegistraGestoreServlet
- Package UtenteManagement
 - UpdateUserProfileServlet
- Package NotizieManagement
 - VisualizzaNotizieServlet
 - CercaNotizieServlet
 - DettaglioNotiziaServlet
- Package SegnalazioneManagement
 - InviaSegnalazioneServlet
 - SuccessoSegnalazioneServlet
 - StoricoSegnalazioneServlet

2.1.3. Storage

La struttura del package include le seguenti classi:

- DatabaseConnection
- Gestore
- GestoreDAO
- Notizia
- NotiziaDAO
- Segnalazione

- SegnalazioneDAO
- Utente
- UtenteDAO

3. CLASS INTERFACES

3.1. UtenteDAO

NOME CLASSE	UtenteDAO
DESCRIZIONE	Metodo per creare un nuovo account utente.
createUser	context UtenteDAO::createUser(user: Utente) : Boolean pre: user <> null and user.nome <> null and user.cognome <> null and user.email <> null and user.passwordHash <> null and user.telefono <> null post: self.getUserByEmail(user.email).nome = user.nome and self.getUserByEmail(user.email).cognome = user.cognome and self.getUserByEmail(user.email).email = user.email and self.getUserByEmail(user.email).telefono = user.telefono
getUserByEmail	Metodo per recuperare un utente tramite email. context UtenteDAO::getUserByEmail(email: String) : Utente pre: email <> null post: (result.email = email and result.nome <> null and result.cognome <> null and result.telefono <> null and result.passwordHash <> null) or (result = null)
emailExists	Metodo per verificare se un'email esiste nel database context UtenteDAO::emailExists(email: String) : Boolean pre: email <> null post: result = true if utente con email esiste, false altrimenti
registraUtente	Metodo per registrare un nuovo utente con password hash. context UtenteDAO::registraUtente(utente: Utente, passwordHash: String) : Boolean pre: utente <> null and utente.nome <> null and utente.cognome <> null and utente.email <> null and utente.telefono <> null and passwordHash <> null post: self.getUserByEmail(utente.email).nome = utente.nome and

	self.getUserByEmail(utente.email).cognome = utente.cognome and self.getUserByEmail(utente.email).email = utente.email and self.getUserByEmail(utente.email).telefono = utente.telefono
authenticateUser	Metodo per autenticare un utente tramite email e password. context UtenteDAO::authenticateUser(email: String, password: String) : Utente pre: email $\diamond$ null and password $\diamond$ null post: (result.email = email and result.passwordHash $\diamond$ null) or (result = null)
getUtenteById	Metodo per recuperare un utente tramite ID. context UtenteDAO::getUtenteById(id: Integer) : Utente pre: id $\diamond$ null post: (result.id = id and result.email $\diamond$ null and result.nome $\diamond$ null and result.cognome $\diamond$ null) or (result = null)
updateUser	Metodo per aggiornare i dati di un utente. context UtenteDAO::updateUser(user: Utente) : Boolean pre: user $\diamond$ null and user.email $\diamond$ null and user.nome $\diamond$ null and user.cognome $\diamond$ null and user.telefono $\diamond$ null post: self.getUserByEmail(user.email).nome = user.nome and self.getUserByEmail(user.email).cognome = user.cognome and self.getUserByEmail(user.email).telefono = user.telefono

3.2. *NotiziaDAO*

NOME CLASSE	NotiziaDAO
DESCRIZIONE	Si occupa dell'accesso e la gestione delle notizie nel database.
inserisciNotizia	Metodo per salvare una nuova notizia. context NotiziaDAO::inserisciNotizia(notizia: Notizia) : Boolean pre: notizia $\diamond$ null and notizia.titolo $\diamond$ null and notizia.descrizione $\diamond$ null and notizia.autore $\diamond$ null post: self.getNotiziaById(notizia.id).titolo = notizia.titolo and

	self.getNotiziaById(notizia.id).descrizione = notizia.descrizione and self.getNotiziaById(notizia.id).autore = notizia.autore
getNotiziaById	Metodo per recuperare una notizia tramite ID. context NotiziaDAO::getNotiziaById(id: Integer) : Notizia pre: id <> null post: (result.id = id and result.titolo <> null and result.descrizione <> null and result.autore <> null and result.stato <> null) or (result = null)
getTutteNotizie	Metodo per recuperare tutte le notizie. context NotiziaDAO::getTutteNotizie() : Collection(Notizia) pre: true post: result->forAll(n n.id <> null and n.titolo <> null and n.stato <> null)
cercaNotizie	Metodo per cercare notizie per keyword (titolo, descrizione o autore). context NotiziaDAO::cercaNotizie(keyword: String) : Collection(Notizia) pre: keyword <> null post: result->forAll(n n.titolo.toLowerCase().contains(keyword.toLowerCase()) or n.descrizione.toLowerCase().contains(keyword.toLowerCase()) or n.autore.toLowerCase().contains(keyword.toLowerCase()))
aggiornaNotizia	Metodo per aggiornare una notizia completa. context NotiziaDAO::aggiornaNotizia(notizia: Notizia) : Boolean pre: notizia <> null and notizia.id <> null and notizia.titolo <> null and notizia.descrizione <> null and notizia.autore <> null post: self.getNotiziaById(notizia.id).titolo = notizia.titolo and self.getNotiziaById(notizia.id).descrizione = notizia.descrizione and self.getNotiziaById(notizia.id).autore = notizia.autore
aggiornaStatoNotizia	Metodo per aggiornare lo stato di una notizia. context NotiziaDAO::aggiornaStatoNotizia(id: Integer, nuovoStato: String) : Boolean pre: id <> null and nuovoStato <> null post: self.getNotiziaById(id).stato = nuovoStato
eliminaNotizia	Metodo per eliminare una notizia. context NotiziaDAO::eliminaNotizia(id: Integer) : Boolean pre: id <> null post: Notizia.allInstances()->forAll(n n.id <> id)

3.3. *SegnalazioneDAO*

NOME CLASSE	SegnalazioneDAO
DESCRIZIONE	Si occupa dell'accesso e la gestione delle segnalazioni nel database.
inserisciSegnalazione	Metodo per salvare una nuova segnalazione. context SegnalazioneDAO::inserisciSegnalazione(segnalazione: Segnalazione, idUtente: Integer) : Boolean pre: segnalazione <> null and segnalazione.titolo <> null and segnalazione.descrizione <> null and segnalazione.url <> null and segnalazione.autore <> null post: self.getSegnalazioneById(segnalazione.id).titolo = segnalazione.titolo and self.getSegnalazioneById(segnalazione.id).descrizione = segnalazione.descrizione and self.getSegnalazioneById(segnalazione.id).stato = 'in_verifica'
getSegnalazioneById	Metodo per recuperare una segnalazione tramite ID. context SegnalazioneDAO::getSegnalazioneById(id: Integer) : Segnalazione pre: id <> null post: (result.id = id and result.titolo <> null and result.descrizione <> null and result.stato <> null) or (result = null)
getSegnalazioniInVerifica	Metodo per recuperare tutte le segnalazioni in attesa di verifica. context SegnalazioneDAO::getSegnalazioniInVerifica() : Collection(Segnalazione) pre: true post: result->forAll(s s.stato = 'in_verifica' and s.id <> null and s.titolo <> null)
getSegnalazioniByUtente	Metodo per recuperare le segnalazioni di un utente specifico. context SegnalazioneDAO::getSegnalazioniByUtente(idUtente: Integer) : Collection(Segnalazione) pre: idUtente <> null post: result->forAll(s s.idUtente = idUtente)

aggiornaStatoSegnalazione	Metodo per aggiornare lo stato di una segnalazione (Verificata o Non Attendibile). context SegnalazioneDAO::aggiornaStatoSegnalazione(idSegnalazione: Integer, nuovoStato: String, idGestore: Integer, idNotizia: Integer) : Boolean pre: idSegnalazione $\neq$ null and nuovoStato $\neq$ null and idGestore $\neq$ null post: self.getSegnalazioneById(idSegnalazione).stato = nuovoStato and self.getSegnalazioneById(idSegnalazione).idGestore = idGestore
getTutteSegnalazioni	Metodo per recuperare tutte le segnalazioni. context SegnalazioneDAO::getTutteSegnalazioni() : Collection(Segnalazione) pre: true post: result->forAll(s s.id $\neq$ null and s.titolo $\neq$ null and s.stato $\neq$ null)
countSegnalazioniByStato	Metodo per ottenere il numero di segnalazioni per stato. context SegnalazioneDAO::countSegnalazioniByStato(stato: String) : Integer pre: stato $\neq$ null post: result $\geq$ 0
getSegnalazioneByIdNotizia	Metodo per recuperare una segnalazione tramite ID notizia. context SegnalazioneDAO::getSegnalazioneByIdNotizia(idNotizia: Integer) : Segnalazione pre: idNotizia $\neq$ null post: (result.idNotizia = idNotizia) or (result = null)

3.4. *GestoreDAO*

NOME CLASSE	GestoreDAO
DESCRIZIONE	Si occupa dell'accesso e la gestione dei gestori (di verifica e tecnici) nel database
login	Metodo per autenticare un gestore tramite email e password hash. context GestoreDAO::login(email: String, passwordHash: String) : Gestore pre: email <> null and passwordHash <> null post: (result.email = email and result.passwordHash = passwordHash) or (result = null)
registraGestore	Metodo per registrare un nuovo gestore (verificatore o tecnico). context GestoreDAO::registraGestore(gestore: Gestore, passwordHash: String, creatoDa: Integer) : Boolean pre: gestore <> null and gestore.nome <> null and gestore.cognome <> null and gestore.email <> null and gestore.telefono <> null and gestore.ruolo <> null and passwordHash <> null post: self.getGestoreById(gestore.id).nome = gestore.nome and self.getGestoreById(gestore.id).cognome = gestore.cognome and self.getGestoreById(gestore.id).email = gestore.email and self.getGestoreById(gestore.id).ruolo = gestore.ruolo
getTuttiGestori	Metodo per recuperare tutti i gestori. context GestoreDAO::getTuttiGestori() : Collection(Gestore) pre: true post: result->forAll(g g.id <> null and g.email <> null and g.ruolo <> null)
getGestoreById	Metodo per recuperare un gestore tramite ID. context GestoreDAO::getGestoreById(id: Integer) : Gestore pre: id <> null post: (result.id = id and result.email <> null and result.nome <> null and result.cognome <> null) or (result = null)
emailEsiste	Metodo per verificare se un'email di gestore esiste nel database. context GestoreDAO::emailEsiste(email: String) : Boolean pre: email <> null post: result = true if gestore con email esiste, false altrimenti